

KeenPay

Agentic commerce with bounded AI

Discount Policy Quick Start Guide

Discount policy quick start

01 September 2026

<https://github.com/ambitiouss22/KeenPay>

Discount Policy Quick Start Guide

5-Minute Setup

1. Apply Database Schema

```
cd /path/to/KeenPay
psql $DATABASE_URL -f docs/DISCOUNT_POLICY_SCHEMA.sql
```

Shell

Verify:

```
psql $DATABASE_URL -c "SELECT * FROM discount_policies LIMIT 1;"
```

Shell

2. Create Initial Policy

```
psql $DATABASE_URL << 'EOF'
-- Example: Hoodie policy
INSERT INTO discount_policies (merchant_id, product_sku, max_discount_pct, daily_budget_paise)
VALUES ('merchant_keen', 'HOODIE-NAVY-M', 25.0, 50000);

-- Get policy_id for segment creation
\set policy_id `psql -tqc "SELECT policy_id FROM discount_policies WHERE product_sku='HOODIE-NAVY-M' LIMIT 1"`

-- Add VIP segment (35% vs 25% global)
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct)
VALUES (:policy_id, 'vip', 35.0);

-- Verify
SELECT * FROM discount_policies WHERE product_sku = 'HOODIE-NAVY-M';
SELECT * FROM discount_segments WHERE user_segment = 'vip';
EOF
```

Shell

3. Run Tests

```
pytest tests/test_discount_policy.py -v

# Expected output: 50+ tests passed ✓
```

Shell

4. Start Application

```
docker-compose up -d
docker logs keenpay_api | grep "Discount"
```

Shell

Core Concepts

Merchant Policy

Defines discount bounds for one product:

```
DiscountPolicy(
    product_sku="HOODIE-NAVY-M",
    max_discount_pct=25.0,          # Global max
    daily_budget_paise=50000,      # Daily spend cap
    per_user_type={
        UserType.VIP: 35.0,        # VIP gets higher
        UserType.BULK_BUYER: 30.0
    }
)
```

Python

Policy in Action

```
Merchant Policy: "Max 25%, VIP gets 35%, Daily budget 500 INR"
      ↓
AI proposes: "Give VIP 40% off"
      ↓
Engine validates: "40% > 35% limit for VIP"
      ↓
Decision: "Approve 35% off instead"
      ↓
User sees: "Your VIP tier qualifies for 35% off"
      ↓
Audit: Record decision in passport + DB
```

Usage in Code

Check Discount Request

```
from api.policy.discount_policy import get_discount_engine, DiscountRequest

engine = get_discount_engine()

decision = engine.check_discount_request(
    DiscountRequest(
        merchant_id="merchant_keen",
        product_sku="HOODIE-NAVY-M",
        user_id="user_vip_123",
        user_type="vip",
        requested_discount_pct=40.0,
        reason="Loyalty program"
    )
)

# Result
print(f"Approved: {decision.approved}") # True
print(f"Amount: {decision.approved_discount_pct}%") # 35.0%
print(f"Reason: {decision.reason}") # "Your tier (vip) gets up to 35% off"
print(f"Rule: {decision.policy_applied}") # "USER_TYPE_LIMIT_vip"
```

Python

In SessionService.negotiate()

```
# Import
from api.policy.discount_policy import get_discount_engine, DiscountRequest

class SessionService:
    def __init__(self):
        self._discount_engine = get_discount_engine()

    async def negotiate(self, state):
        # ... AI proposes discount ...
        proposed_discount_pct = 40.0

        # Check policy
        decision = self._discount_engine.check_discount_request(
            DiscountRequest(
                merchant_id=state.merchant_id,
                product_sku=state.selected_line_items[0]["sku"],
                user_id=state.user_id,
                user_type=self._categorize_user_type(state.user_id),
                requested_discount_pct=proposed_discount_pct
            )
        )

        # Use approved discount
        state.proposed_offer["discount_pct"] = decision.approved_discount_pct
        state.proposed_offer["reason"] = decision.reason

        # Record in audit + passport
        await self._db.discount_decisions.insert_one(decision.to_dict())
        await self._passport_engine.add_entry(
            event_type="DISCOUNT_POLICY_DECISION",
            payload=decision.to_dict()
        )

        return state
```

Python

Common Tasks

Create Policy for New Product

```
psql $DATABASE_URL << 'EOF'
-- Define policy
INSERT INTO discount_policies
(merchant_id, product_sku, max_discount_pct, daily_budget_paise, description)
VALUES (
    'merchant_keen',
    'NEW-PRODUCT-SKU',
    20.0,
    100000,
    'New product: 20% max, 1000 INR/day'
);

-- Get policy ID
SELECT policy_id FROM discount_policies WHERE product_sku = 'NEW-PRODUCT-SKU';
EOF
```

Shell

Add VIP Override

```
psql $DATABASE_URL << 'EOF'
-- Get policy_id
SELECT p.policy_id FROM discount_policies p WHERE p.product_sku = 'HOODIE-NAVY-M';

-- Add VIP segment
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct)
VALUES ('[POLICY_ID_FROM_ABOVE]', 'vip', 35.0);
EOF
```

Shell

Check Today's Budget Usage

```
psql $DATABASE_URL << 'EOF'
SELECT
    p.product_sku,
    p.daily_budget_paise,
    t.daily_used_paise,
    p.daily_budget_paise - t.daily_used_paise as remaining,
    ROUND(100.0 * t.daily_used_paise / p.daily_budget_paise, 1) as utilization_pct
FROM discount_usage_tracking t
JOIN discount_policies p ON t.policy_id = p.policy_id
WHERE t.tracking_date = CURRENT_DATE
ORDER BY utilization_pct DESC;
EOF
```

Shell

View All Discount Decisions Today

```
psql $DATABASE_URL << 'EOF'
SELECT
    product_sku,
    user_segment,
    COUNT(*) as decisions,
    SUM(CASE WHEN status='APPROVED' THEN 1 ELSE 0 END) as approved,
    SUM(CASE WHEN status='DENIED' THEN 1 ELSE 0 END) as denied,
    ROUND(AVG(approved_discount_pct), 2) as avg_approved_pct
FROM discount_decisions
WHERE created_at > CURRENT_DATE
GROUP BY product_sku, user_segment;
EOF
```

Shell

Reset Daily Budget (for testing)

```
psql $DATABASE_URL << 'EOF'
-- Get policy ID
SELECT p.policy_id FROM discount_policies p WHERE p.product_sku = 'HOODIE-NAVY-M';

-- Reset for today
DELETE FROM discount_usage_tracking
WHERE policy_id = '[POLICY_ID]' AND tracking_date = CURRENT_DATE;

-- Or via function
SELECT reset_daily_discount_budget('[POLICY_ID']::UUID, CURRENT_DATE);
EOF
```

Shell

API Endpoints

Create/Update Policy

```
curl -X POST http://localhost:8000/api/v1/discount-policies \
-H "Authorization: Bearer merchant_token" \
-H "Content-Type: application/json" \
-d '{
  "product_sku": "HOODIE-NAVY-M",
  "max_discount_pct": 25,
  "daily_budget_paise": 50000,
  "segment_limits": [
    {"user_segment": "vip", "max_discount_pct": 35},
    {"user_segment": "bulk_buyer", "max_discount_pct": 30}
  ],
  "description": "Hoodie: up to 25% off"
}'
```

Shell

Get Policy

```
curl http://localhost:8000/api/v1/discount-policies/HOODIE-NAVY-M
```

Shell

Get Analytics

```
curl http://localhost:8000/api/v1/discount-policies/analytics/HOODIE-NAVY-M
```

Shell

Testing

Unit Tests

```
# Run all discount policy tests
pytest tests/test_discount_policy.py -v

# Run specific test
pytest tests/test_discount_policy.py::test_no_policy_denies_discount -v

# Run with coverage
pytest tests/test_discount_policy.py --cov=api.policy.discount_policy
```

Shell

Manual Test Flow

```
# 1. Create session
curl -X POST http://localhost:8000/api/v1/sessions \
  -H "Content-Type: application/json" \
  -d '{"merchant_id": "merchant_keen", "user_id": "user_vip"}'

# 2. Start negotiation (triggers negotiate node)
curl -X POST http://localhost:8000/api/v1/sessions/[session_id]/negotiate \
  -H "Content-Type: application/json" \
  -d '{"proposed_discount_pct": 50}'

# 3. Check decision
curl http://localhost:8000/api/v1/sessions/[session_id]/discount-decision

# 4. Verify audit
psql $DATABASE_URL -c "SELECT * FROM discount_decisions WHERE session_id = '[session_id]';"
```

Shell

Monitoring

Key Metrics Dashboard

```
# 1. Policy effectiveness
psql $DATABASE_URL << 'EOF'
SELECT * FROM discount_policy_analytics
WHERE merchant_id = 'merchant_keen'
ORDER BY total_decisions DESC;
EOF

# 2. Budget status
psql $DATABASE_URL << 'EOF'
SELECT
  p.product_sku,
  ROUND(100.0 * t.daily_used_paise / p.daily_budget_paise, 1) as utilization_pct,
  t.daily_used_paise,
  p.daily_budget_paise - t.daily_used_paise as remaining_paise
FROM discount_usage_tracking t
JOIN discount_policies p ON t.policy_id = p.policy_id
WHERE t.tracking_date = CURRENT_DATE
ORDER BY utilization_pct DESC;
EOF

# 3. Denial rate
psql $DATABASE_URL << 'EOF'
SELECT
  product_sku,
  COUNT(*) as total,
  SUM(CASE WHEN status='DENIED' THEN 1 ELSE 0 END) as denied,
  ROUND(100.0 * SUM(CASE WHEN status='DENIED' THEN 1 ELSE 0 END) / COUNT(*), 1) as denial_rate_pct
FROM discount_decisions
WHERE created_at > NOW() - INTERVAL '24 hours'
GROUP BY product_sku;
EOF
```

Shell

Alerts

```
-- Alert: High denial rate
SELECT product_sku
FROM discount_decisions
WHERE created_at > NOW() - INTERVAL '6 hours'
GROUP BY product_sku
HAVING SUM(CASE WHEN status='DENIED' THEN 1 ELSE 0 END)::float / COUNT(*) > 0.2
ORDER BY COUNT(*) DESC;

-- Alert: Budget nearly exhausted
SELECT p.product_sku, ROUND(100.0 * t.daily_used_paise / p.daily_budget_paise, 1) as utilization_pct
FROM discount_usage_tracking t
JOIN discount_policies p ON t.policy_id = p.policy_id
WHERE t.tracking_date = CURRENT_DATE AND t.daily_used_paise / p.daily_budget_paise > 0.9;

-- Alert: Unusual discount requests
SELECT product_sku, user_segment, requested_discount_pct, COUNT(*) as count
FROM discount_decisions
WHERE created_at > NOW() - INTERVAL '1 hour'
GROUP BY product_sku, user_segment, requested_discount_pct
HAVING COUNT(*) > 10 AND requested_discount_pct > 50
ORDER BY count DESC;
```

SQL

Troubleshooting

Policy Not Applied

```
# 1. Check policy exists
psql $DATABASE_URL -c "SELECT * FROM discount_policies WHERE product_sku = 'YOUR-SKU';"

# 2. Check policy is active
psql $DATABASE_URL -c "SELECT * FROM discount_policies WHERE product_sku = 'YOUR-SKU' AND is_active = TRUE;"

# 3. Check engine is initialized in SessionService
grep -n "self._discount_engine" api/services/session.py

# 4. Check negotiate node calls engine
grep -A5 "check_discount_request" api/services/session.py
```

Shell

Decision Not Recorded

```
# 1. Check audit logging is enabled
psql $DATABASE_URL -c "SELECT * FROM discount_decisions LIMIT 1;"

# 2. Check passport entry creation
psql $DATABASE_URL -c "SELECT * FROM passport_entries WHERE event_type='DISCOUNT_POLICY_DECISION' LIMIT 1;"

# 3. Check for errors in logs
docker logs keenpay_api | grep -i discount
```

Shell

Budget Not Resetting

```
# 1. Check tracking table
psql $DATABASE_URL -c "SELECT * FROM discount_usage_tracking WHERE tracking_date = CURRENT_DATE;"

# 2. Manually reset (for testing)
psql $DATABASE_URL -c "UPDATE discount_usage_tracking SET daily_used_paise = 0 WHERE tracking_date = CURRENT_DATE;"

# 3. Check scheduled reset function is running
psql $DATABASE_URL -c "SELECT * FROM pg_stat_user_functions WHERE funcname LIKE '%discount%';"
```

Shell

Performance Considerations

Indexes

Discount policy module uses these indexes for performance:

```
CREATE INDEX idx_discount_policies_merchant ON discount_policies (merchant_id, is_active);
CREATE INDEX idx_discount_decisions_policy ON discount_decisions (policy_id, created_at DESC);
CREATE INDEX idx_discount_usage_tracking_policy ON discount_usage_tracking (policy_id);
```

SQL

Query Performance

All policy checks happen in memory (in-process engine). Database queries only for:

- Initial policy load (cached)
- Audit logging (non-blocking)
- Analytics queries (background)

Result: Discount policy decision <10ms per request.

Files Reference

File	Purpose	Lines
<code>api/policy/discount_policy.py</code>	Core engine	253
<code>docs/DISCOUNT_POLICY_SCHEMA.sql</code>	Database schema	450
<code>docs/DISCOUNT_POLICY_INTEGRATION.md</code>	Integration guide	500+
<code>docs/DISCOUNT_POLICY_SAFETY_FIX.md</code>	Architecture + FAQ	600+
<code>tests/test_discount_policy.py</code>	Unit tests	600+
<code>docs/DISCOUNT_POLICY_QUICK_START.md</code>	This file	400+

Next Steps

- 1 ✓ Apply schema - `psql $DATABASE_URL -f docs/DISCOUNT_POLICY_SCHEMA.sql`
- 2 ✓ Create policies - Define bounds for each product
- 3 ✓ Run tests - `pytest tests/test_discount_policy.py -v`
- 4 ✓ Integrate - Update `SessionService.negotiate()` to call engine
- 5 ✓ Deploy - Push changes and deploy

- 6 ✓ Monitor - Track decisions table + analytics view
 - 7 ✓ Iterate - Adjust policies based on data
-

Support

- Integration help → See `docs/DISCOUNT_POLICY_INTEGRATION.md`
- Architecture details → See `docs/DISCOUNT_POLICY_SAFETY_FIX.md`
- Test examples → See `tests/test_discount_policy.py`
- Database queries → See examples in this file

Problem? Check logs:

```
docker logs keenpay_api | grep -i "discount\|policy"
psql $DATABASE_URL -c "SELECT * FROM discount_decisions ORDER BY created_at DESC LIMIT 10;"
```

Shell